

SaaS de Gestión de Créditos — SRS

Especificación de Requerimientos de Software

Versión 1.2 · 15 de septiembre de 2026 · Documento de trabajo interno

BASE DE REQUERIMIENTOS DEL PROYECTO

SAAS DE GESTIÓN DE CRÉDITOS Y COBRO DIARIO

Documento de Especificación de Requerimientos de Software (SRS)

Versión 1.2 — Documento base de definición (nombres en español + calidad)

Preparado para: uso interno, planificación del producto y equipo de desarrollo

Fecha: 15 de septiembre de 2026

DOCUMENTO DE TRABAJO. Este documento formaliza la definición inicial del producto. Las decisiones marcadas como *por definir* deben confirmarse antes de convertirlas en compromisos de implementación.

Cambios respecto a v1.0: frontend Angular; monorepo dockerizado desde el inicio; Swagger UI y colecciones Postman como criterio de desarrollo; esquema de base de datos formalizado (entidades, relaciones, campos, índices y transacciones). v1.2: nombres de tablas/columnas y código de dominio en español; tests automatizados como puerta de entrega.

Tabla de contenido

1. Definición general del producto
2. Tipos de usuarios y roles
3. Flujo principal del producto
4. Requerimientos funcionales — Prestamista / Administrador
5. Requerimientos funcionales — Cobrador
6. Requerimientos funcionales — Administración de plataforma
7. Planes comerciales
8. Requerimientos no funcionales
9. Entorno de desarrollo y calidad de API
10. Reglas de negocio
11. Casos de uso
12. Flujos del sistema
13. Arquitectura funcional
14. Modelo de datos
15. Seguridad y auditoría
16. Escalabilidad
17. MVP vs. V1 vs. futuras versiones
18. Roadmap
19. Criterios de aceptación y pendientes

1. Definición general del producto

1.1 Nombre de trabajo

SaaS de Gestión de Créditos y Cobro Diario (nombre comercial por definir).

1.2 Descripción

Plataforma web SaaS orientada a personas independientes y pequeños negocios que administran préstamos y realizan cobros diarios dentro de una ciudad. Centraliza clientes, créditos, cuotas, cobradores, recaudos, saldos, cartera, mora, reportes y control administrativo.

1.3 Contexto operativo identificado

- La operación objetivo puede manejar aproximadamente entre 5 y 15 cobradores.
- Cada cobrador puede administrar aproximadamente entre 60 y 100 clientes.
- El escenario de referencia de mayor tamaño considerado es de hasta 15 cobradores.
- El producto se plantea inicialmente como aplicación web responsive (Angular PWA), no como aplicación móvil nativa.
- El cobrador debe poder utilizar la plataforma cómodamente desde un teléfono durante su recorrido diario.
- La infraestructura debe estar preparada para crecer con la cantidad de negocios y usuarios.
- El desarrollo se ejecuta desde el día uno en un **monorepo dockerizado** que contiene frontend Angular y backend API.

1.4 Problema que resuelve

- Información dispersa entre cuadernos, hojas de cálculo, mensajes y otros medios.
- Dificultad para saber cuánto debe cobrar cada cobrador durante el día.
- Dificultad para controlar saldos, cuotas pagadas, cuotas pendientes y cartera vencida.
- Poca visibilidad del desempeño individual de los cobradores.
- Errores humanos al calcular o registrar operaciones.
- Dificultad para consolidar la operación de varios cobradores en un único lugar.

1.5 Solución propuesta

Cliente → Crédito → Plan de cuotas → Asignación a cobrador → Cobro diario → Registro del pago → Actualización de saldo → Cartera → Reportes y control administrativo.

1.6 Propuesta de valor

"Controla tus créditos y cobros diarios desde un solo lugar, con información clara para quien administra y una experiencia rápida para quien cobra."

1.7 Alcance conceptual

- Gestión de múltiples clientes por prestamista.
- Gestión de múltiples cobradores por operación.
- Asignación de clientes y/o cartera a cobradores.
- Creación y seguimiento de créditos.
- Generación del calendario o plan de cuotas.
- Registro de cobros diarios.
- Consulta de saldos y estado de cartera.
- Dashboard y reportes administrativos.
- Roles y permisos.
- Suscripción SaaS y control de plan.

- Auditoría de operaciones relevantes.
- Diseño responsive para celular, tablet y computador (Angular).

1.8 Fuera del alcance inicial / por definir

- Aplicación móvil nativa Android/iOS: se prioriza web responsive/PWA.
- Pasarela para que el deudor pague directamente dentro de la plataforma: por definir.
- Integraciones bancarias y conciliación automática: por definir.
- Geolocalización del cobrador: por definir y sujeta a validación de valor y privacidad.
- WhatsApp/SMS automáticos: por definir.
- Calificación crediticia automática: por definir.
- Desembolso digital: por definir.

2. Tipos de usuarios y roles

Rol	Descripción	Acceso principal
Propietario / Prestamista	Dueño de la operación.	Panel completo de su negocio
Administrador del negocio	Apoya la gestión operativa y administrativa.	Clientes, créditos, cartera, reportes y usuarios según permisos
Cobrador	Realiza y registra cobros diarios.	Cartera asignada y agenda de cobro
Soporte de plataforma	Personal interno del proveedor SaaS.	Soporte e incidencias con mínimo privilegio
Administrador de plataforma	Responsable interno del SaaS.	Negocios, suscripciones, configuración, auditoría y operación

Aislamiento: cada negocio cliente debe ver únicamente la información de su propia organización. El sistema debe ser multiempresa/multitenant desde el diseño.

Permisos: un cobrador no puede administrar configuraciones ni consultar cartera que no le haya sido asignada.

3. Flujo principal del producto

CLIENTE → CRÉDITO → PLAN DE CUOTAS → ASIGNACIÓN → AGENDA DE COBRO → COBRO DIARIO → REGISTRO → ACTUALIZACIÓN DE SALDO → CARTERA → REPORTES → CONTROL ADMINISTRATIVO

Flujo operativo:

- El prestamista registra al cliente.
- Se crea un crédito con monto, condiciones y periodicidad.
- El sistema genera el plan de cuotas según las reglas configuradas.
- El crédito se asigna a un cobrador.
- El cobrador consulta su cartera y obligaciones del día.
- Registra el pago recibido.
- El sistema valida y actualiza cuota, saldo y estado **en una sola transacción**.
- La operación queda trazada.
- El administrador consulta recaudo, cartera, mora y desempeño.
- Los atrasos aparecen como mora según las reglas configuradas.

Nota sobre tasas: durante el descubrimiento se mencionó que existen operaciones con porcentajes altos, incluso un ejemplo del 20%. El sistema debe parametrizar las condiciones, pero su uso debe someterse a validación legal y

financiera antes de producción.

4. Requerimientos funcionales — Prestamista / Administrador

Estos requerimientos representan el núcleo funcional del negocio cliente.

ID	Requerimiento	Descripción	Prioridad
RF-001	Registro e inicio de sesión	Crear cuenta, iniciar/cerrar sesión y recuperar acceso.	MVP
RF-002	Configuración del negocio	Datos operativos, moneda, parámetros y preferencias.	MVP
RF-003	Gestión de clientes	Crear, consultar, editar, activar/inactivar y buscar.	MVP
RF-004	Gestión de cobradores	Crear usuarios, asignarlos y administrar su estado.	MVP
RF-005	Gestión de créditos	Registrar monto, condiciones, fecha y estado.	MVP
RF-006	Plan de cuotas	Generar y consultar cuotas según periodicidad y condiciones.	MVP
RF-007	Asignación de cartera	Asignar clientes/créditos a cobradores.	MVP
RF-008	Control de cobro diario	Consultar obligaciones del día y registrar recaudos.	MVP
RF-009	Historial de pagos	Consultar pagos asociados a crédito.	MVP
RF-010	Control de cartera	Ver cartera vigente, pagada, pendiente y mora.	MVP
RF-011	Dashboard	Indicadores de cartera y recaudo.	MVP
RF-012	Reportes	Consultar y exportar información operativa.	V1
RF-013	Gestión de mora	Identificar atrasos y hacer seguimiento.	MVP
RF-014	Auditoría	Registrar acciones relevantes sobre datos críticos.	MVP
RF-015	Gestión de suscripción	Consultar plan, límites, estado y facturación.	MVP
RF-016	Notificaciones	Avisos operativos de cobros, vencimientos y eventos.	V1
RF-017	Configuración avanzada	Parámetros de negocio sin cambiar código.	V1

Notas:

- **RF-005:** las condiciones originales de un crédito deben conservarse en `creditos.condiciones_originales` y no cambiar retroactivamente por modificar configuraciones posteriores.
- **RF-008:** el flujo de cobro debe minimizar pasos y escritura.

- **RF-014:** creación/anulación de pagos, cambios de crédito y reasignaciones deben quedar trazados en auditorias.

5. Requerimientos funcionales — Cobrador

ID	Requerimiento	Descripción	Prioridad
RC-001	Inicio de sesión	Acceso seguro al módulo operativo.	MVP
RC-002	Mi cartera	Visualizar solo clientes/créditos asignados.	MVP
RC-003	Cobros del día	Ver obligaciones programadas para la fecha.	MVP
RC-004	Ficha rápida	Consultar datos mínimos del cliente/crédito.	MVP
RC-005	Registrar pago	Registrar monto y fecha/hora.	MVP
RC-006	Confirmación	Confirmar operación antes/después del registro.	MVP
RC-007	Historial	Consultar pagos anteriores.	MVP
RC-008	Pendientes	Identificar cobros no realizados y atrasos.	MVP
RC-009	Resumen diario	Total esperado, cobrado y pendiente.	MVP
RC-010	Observaciones	Registrar novedades de cobro.	V1
RC-011	Comprobante	Mostrar/generar comprobante digital.	V1

Principio UX: la pantalla principal debe responder rápidamente a "qué cobro hoy", "a quién", "cuánto" y "qué ya cobré". Botones grandes, navegación corta, tablas adaptadas a pantallas pequeñas y tolerancia a conexiones inestables serán criterios de diseño del frontend Angular.

6. Requerimientos funcionales — Administración de plataforma

ID	Requerimiento	Descripción	Prioridad
RA-001	Acceso administrativo	Acceso separado y protegido.	MVP
RA-002	Gestión de negocios	Consultar, activar, suspender y administrar cuentas.	MVP
RA-003	Gestión de usuarios	Buscar y administrar usuarios por negocio.	MVP
RA-004	Planes y límites	Crear/editar planes y límites.	MVP
RA-005	Suscripciones	Consultar estado, pagos y cancelaciones.	MVP
RA-006	Monitoreo	Consultar salud, errores y eventos.	V1

ID	Requerimiento	Descripción	Prioridad
RA-007	Auditoría	Consultar acciones administrativas.	MVP
RA-008	Configuración global	Parámetros generales.	V1
RA-009	Soporte	Atender incidencias con permisos limitados.	V1
RA-010	Métricas de negocio	Usuarios, clientes SaaS, ingresos y consumo.	V1

El panel del proveedor debe estar separado lógicamente del panel operativo de los clientes.

7. Planes comerciales

Los precios son una hipótesis comercial inicial definida durante el descubrimiento y deben validarse con costos reales, mercado, impuestos y soporte.

Plan	Implementación	Mensualidad	Cobradores	Perfil
Emprendedor	\$1.250.000	\$250.000/mes	Hasta 3	Prestamista pequeño
Profesional	\$2.500.000	\$425.000/mes	Hasta 10	Operación con varios cobradores
Empresarial	\$3.500.000	\$750.000/mes	Hasta 15	Operación de mayor volumen

- **Emprendedor:** clientes, créditos, cuotas, cobros diarios, historial, cartera, dashboard básico, soporte y actualizaciones.
- **Profesional:** todo lo anterior + reportes avanzados, control por cobrador, exportaciones y soporte prioritario.
- **Empresarial:** todo lo anterior + múltiples administradores, indicadores, configuración personalizada y soporte prioritario.

El modelo principal es implementación + suscripción; no se plantea vender el código como producto definitivo.

8. Requerimientos no funcionales

Categoría	Requerimiento
Disponibilidad	Meta inicial de referencia: 99,5% mensual, excluyendo mantenimiento programado.
Rendimiento	Operaciones de clientes, créditos y cobros deben responder rápidamente.
Responsive	Correcto funcionamiento en teléfonos, tablets y computadores (Angular PWA).
Escalabilidad	Crecimiento de negocios y usuarios sin rediseñar el dominio.
Seguridad	HTTPS, sesiones seguras, control de acceso, cifrado y protección de datos.
Aislamiento multiempresa	Un negocio no puede consultar ni modificar datos de otro (negocio_id en cada consulta protegida).
Calidad de entrega	Todo cambio de comportamiento incluye tests automatizados; sin verde no se entrega.

Categoría	Requerimiento
Auditoría	Operaciones críticas registran usuario, fecha, acción y recurso.
Usabilidad	Cobro diario con el menor número razonable de pasos.
Mantenibilidad	Frontend Angular, API, persistencia y workers claramente separados dentro de un monorepo.
Observabilidad	Errores y eventos críticos monitoreables.
Backups	Copias y procedimiento de restauración probado.
Reproducibilidad	El entorno de desarrollo se levanta con Docker Compose; no se admite "funciona en mi máquina" como estándar.

9. Entorno de desarrollo y calidad de API

Estos requerimientos son **obligatorios desde el MVP-1**. Un módulo backend no se considera terminado si no cumple esta sección.

9.1 Monorepo dockerizado

El repositorio único `creditos-saas` contiene frontend Angular (`apps/web`), API (`apps/api`) y workers (`apps/workers`). Cada aplicación tiene su `Dockerfile`. Docker Compose orquesta `web` + `api` + `workers` + PostgreSQL en local.

Cualquier desarrollador debe poder levantar el entorno completo con un único comando de Compose.

9.2 Swagger UI / OpenAPI

La API expone documentación OpenAPI navegable (Swagger UI) en el ambiente de desarrollo.

- Cada endpoint nuevo o modificado se documenta (path, schemas, roles, códigos HTTP, ejemplos).
- Los ejemplos no contienen secretos reales.
- Swagger UI es la referencia viva para frontend y pruebas manuales.

9.3 Colecciones Postman

Las pruebas de desarrollador de endpoints se realizan con colecciones Postman versionadas en el monorepo (`tests/postman/`), organizadas por módulo y con entornos `local` / `staging`.

- Variables `{{url_base}}`, `{{token}}`, `{{negocio_id}}`, IDs de ejemplo.
- Carpeta por módulo (Auth, Clientes, Créditos, Cobros, Cartera, Admin SaaS).
- Un endpoint nuevo no se mergea sin request Postman equivalente.

9.4 Tests automatizados (puerta de entrega)

El desarrollo terminado es código + Swagger/Postman + **tests en verde**. Si la suite del cambio está en rojo, la actividad no se entrega.

- API: Vitest junto al servicio + pruebas HTTP (éxito y 403 de otro negocio).
- Flujos de dinero: transacción, anulación, cartera no asignada, saldo no negativo.
- Angular: specs de componente/servicio del cambio.
- Comando raíz `npm test`. Nomenclatura de tests en español (debe ...).
- Detalle SDD: workflow `calidad-tests`.

9.5 Stack de referencia (detalle en el documento de arquitectura)

- Frontend: **Angular** + TypeScript (SPA responsive / PWA).
- Backend: Node.js + TypeScript (NestJS recomendado).

- BD: PostgreSQL.
- Contenerización: Docker + Docker Compose.

10. Reglas de negocio

- Cada cliente pertenece a un negocio y no es visible desde otro negocio.
- Cada cobrador pertenece a un negocio y solo accede a cartera asignada (`asignaciones activas`).
- Un crédito conserva sus condiciones históricas (`condiciones originales`).
- Registrar un pago actualiza cuota/saldo/estado de forma consistente (transacción atómica sobre pagos, cuotas, créditos y auditorías).
- La anulación o corrección de un pago debe dejar trazabilidad; el pago no se borra, se anula.
- La cartera debe distinguir estados como pendiente/vigente, pagada y mora; catálogo exacto por definir, valores iniciales en el esquema de BD.
- La asignación y reasignación de cartera debe quedar auditada; solo una asignación activa por crédito.
- Los límites de cobradores dependen del plan contratado (`planes.limite_cobradores`).
- Los cambios de plan y prorrateos deben definirse comercialmente.
- Una suscripción suspendida debe restringir funciones según política, conservando información conforme a retención.
- Las tasas, intereses y recargos deben validarse legalmente antes de producción.
- La plataforma se plantea como proveedor tecnológico y no como prestamista.

11. Casos de uso

CU-001 — Registrar cliente: administrador → crear cliente → validar documento único por negocio → guardar → cliente disponible para crédito.

CU-002 — Crear crédito: seleccionar cliente → registrar monto/condiciones → generar cuotas → persistir snapshot de condiciones → confirmar → activar.

CU-003 — Asignar cartera: consultar cartera → seleccionar crédito → seleccionar cobrador → confirmar → cerrar asignación previa si existe → registrar asignación activa.

CU-004 — Registrar cobro: cobrador → cobros del día → cliente → obligación → registrar pago → validar → actualizar saldo → auditar → confirmar.

CU-005 — Consultar cartera: administrador → filtros → cartera → detalle/exportación según permisos.

CU-006 — Gestionar suscripción: plan → checkout → pago → webhook → activar/actualizar límites.

CU-007 — Incidencia: soporte → negocio → consultar eventos permitidos → resolver → registrar acción.

12. Flujos del sistema

12.1 Alta del negocio: Landing → Registro → autenticación → creación de negocio → selección de plan → pago/activación → configuración → dashboard.

12.2 Cliente y crédito: Cliente → validación → crédito → condiciones → cuotas → activación → asignación.

12.3 Jornada del cobrador: Login → Mi cartera → Cobros de hoy → Cliente → Cuota → Registrar pago → Confirmar → Actualizar saldo → siguiente cliente.

12.4 Mora: cuota no pagada → atraso → estado de mora → cartera vencida → seguimiento → pago/gestión posterior.

12.5 Cierre del día: cobrador consulta esperado/cobrado/pendiente → administrador consulta consolidado → historial.

12.6 Suscripción: plan → checkout → pago → webhook → activar/actualizar → aplicar límites.

12.7 Error: falla → evitar estado parcial → registrar error → mensaje comprensible → reintento o soporte.

13. Arquitectura funcional

Módulo	Responsabilidad
Autenticación	Identidad, sesiones y control de acceso.
Tenant/Negocio	Aislamiento y configuración.
Clientes	Ficha y ciclo de vida.
Créditos	Condiciones, estados y cuotas.
Cobranza	Agenda diaria y pagos.
Cartera	Saldos, estados y mora.
Usuarios/Roles	Cobradores, administradores y permisos.
Reportes	Indicadores, consultas y exportaciones.
Suscripciones	Planes, límites, pagos y estado.
Auditoría	Trazabilidad.
Administración SaaS	Negocios, planes y soporte.
Documentación de API	OpenAPI / Swagger UI + colecciones Postman.

La separación es funcional; no obliga a crear microservicios en el MVP. El despliegue de desarrollo es un monorepo dockerizado.

Frontend: aplicación Angular (administrador en escritorio, cobrador en móvil). Backend: API modular + workers para procesos asíncronos. Persistencia: PostgreSQL.

14. Modelo de datos

Modelo conceptual:

NEGOCIO (1) → (N) USUARIOS; NEGOCIO (1) → (N) CLIENTES; CLIENTE (1) → (N) CRÉDITOS; CRÉDITO (1) → (N) CUOTAS; CUOTA (1) → (N) PAGOS; CRÉDITO (1) → (N) ASIGNACIONES → (N):(1) COBRADOR; NEGOCIO (1) → (1) SUSCRIPCIÓN; SUSCRIPCIÓN (N) → (1) PLAN; OPERACIÓN (1) → (N) AUDITORÍA.

Fuente canónica detallada: docs/00-esquema-bd.md. Resumen de entidades:

Entidad	Tabla	Propósito
Negocio	negocios	Cliente empresarial del SaaS
Usuario	usuarios	Prestamista, administrador, cobrador o rol de plataforma
Cliente	clientes	Persona asociada a la operación crediticia
Crédito	creditos	Obligación registrada; condiciones_originales inmutable
Cuota	cuotas	Unidad programada del plan
Pago	pagos	Recaudo aplicado a una cuota

Entidad	Tabla	Propósito
Asignación	asignaciones	Relación cartera-cobrador (una activa por crédito)
Plan	planes	Configuración comercial global
Suscripción	suscripciones	Estado del servicio (1:1 con negocio)
Auditoría	auditorias	Acciones críticas, append-only
Notas de cobro	notas_cobro	V1 — RC-010
Comprobantes	comprobantes_pago	V1 — RC-011; archivo en object storage

Nombres de tablas, columnas y código de dominio en **español** (docs/00-esquema-bd.md v1.2).

14.1 Campos por entidad (MVP)

negocios: id UUID PK; nombre_comercial; razon_social (nullable); documento_fiscal (nullable); moneda default COP; estado (activo/suspendido/cancelado); configuracion JSONB; fecha_creacion; fecha_actualizacion.

usuarios: id UUID PK; negocio_id FK nullable (nulo en roles de plataforma); nombre; correo unique; hash_contrasena; rol (propietario/administrador/cobrador/soporte/admin_plataforma); estado; telefono nullable; fecha_creacion; ultimo_acceso; fecha_actualizacion. CHECK: roles de negocio exigen negocio_id; roles de plataforma lo prohíben.

clientes: id UUID PK; negocio_id FK; nombre_completo; tipo_documento; numero_documento; telefono; direccion nullable; referencia_ubicacion nullable; estado; fecha_creacion; fecha_actualizacion; creado_por FK usuarios. Unique (negocio_id, tipo_documento, numero_documento).

creditos: id UUID PK; negocio_id FK; cliente_id FK; monto_principal NUMERIC(18,2) > 0; tasa_interes NUMERIC(8,4) >= 0; periodicidad (diaria/semanal/quincenal/mensual); numero_cuotas >= 1; fecha_desembolso; estado (activo/pagado/mora/anulado/refinanciado); condiciones_originales JSONB immutable; fecha_creacion; fecha_actualizacion; creado_por FK usuarios.

cuotas: id UUID PK; negocio_id FK; credito_id FK; numero_cuota; fecha_vencimiento; monto_esperado; saldo_pendiente; estado (pendiente/pagada/parcial/mora/anulada). Unique (credito_id, numero_cuota).

pagos: id UUID PK; negocio_id FK; cuota_id FK; credito_id FK; cobrador_id FK usuarios; monto > 0; fecha_pago; metodo_pago; estado (valido/anulado); motivo_anulacion nullable; anulado_por nullable; fecha_anulacion nullable; fecha_creacion. Un pago válido no se edita: se anula.

asignaciones: id UUID PK; negocio_id FK; credito_id FK; cobrador_id FK usuarios; fecha_asignacion; fecha_fin nullable; estado (activa/finalizada); asignado_por FK usuarios. Unique parcial: una asignación activa por crédito.

planes: id UUID PK; codigo unique (empreendedor/profesional/empresarial); nombre; limite_cobradores (3/10/15); precio_implementacion; precio_mensual; caracteristicas JSONB; estado; fecha_creacion.

suscripciones: id UUID PK; negocio_id unique FK; plan_id FK; estado (activa/pago_fallido/cancelada/suspendida); fecha_inicio; fecha_renovacion; fecha_cancelacion nullable; referencia_pago_externo nullable; timestamps.

auditorias: id UUID PK; negocio_id nullable; usuario_id FK; entidad; entidad_id; accion; detalle JSONB; fecha; ip_origen nullable. Append-only.

14.2 Integridad

- Toda consulta de negocio filtra por negocio_id.
- Registrar pago, anular pago y reasignar cartera son transacciones atómicas con auditoría.
- Comprobantes (V1) no se guardan como binario en PostgreSQL.

15. Seguridad y auditoría

- HTTPS obligatorio.

- Contraseñas nunca en texto plano.
- Secretos fuera del código y repositorio (variables de entorno por ambiente, incluyendo Docker Compose).
- Autorización por rol y negocio (`negocio_id`) en cada operación protegida.
- Auditoría de pagos, crédito, cartera y acciones administrativas.
- Backups cifrados y procedimiento de restauración.
- Protección contra abuso de autenticación y endpoints sensibles.
- Panel SaaS separado del panel de clientes.
- Política de tratamiento de datos y términos validados antes de producción.
- Ejemplos de Swagger/Postman con placeholders, nunca credenciales reales.

16. Escalabilidad

El escenario de referencia no exige infraestructura masiva inicialmente. Se debe evitar, sin embargo, cualquier diseño que impida crecer.

- Multiempresa desde el inicio.
- Base relacional con índices adecuados (cartera del día, recaudo, asignación activa).
- API stateless para escalar horizontalmente.
- Procesos asíncronos para reportes, notificaciones y tareas programadas (`apps/workers`).
- Backups y observabilidad desde el MVP.
- Capacidad de aumentar recursos sin cambiar el dominio.
- Separación futura de procesos de alto consumo cuando el volumen lo justifique.
- Docker como base de desarrollo y de futuros pipelines CI/CD.

17. MVP vs. V1 vs. futuras versiones

Etapas	Alcance
MVP	Monorepo dockerizado, Angular, Swagger UI, Postman, negocios, usuarios, cobradores, clientes, créditos, cuotas, asignación, cobro diario, historial, cartera, mora básica, dashboard, roles, auditoría y suscripción.
V1	Reportes avanzados, exportaciones, notificaciones, comprobantes, observaciones de cobro, configuración avanzada, UX móvil refinada, monitoreo y métricas.
V2	Pagos integrados, conciliación, automatizaciones, geolocalización si aporta valor, comunicaciones e integraciones.
V3	API para terceros, automatización avanzada y capacidades de mayor escala.

La prioridad es validar el flujo completo de cobranza antes de añadir funcionalidades periféricas.

18. Roadmap

Fase	Objetivo
Descubrimiento	Cerrar reglas, campos, estados, cálculos, operación y requisitos legales.

Fase	Objetivo
Diseño	UX Angular responsive, arquitectura, datos, roles, seguridad y OpenAPI.
MVP-1	Compose + Angular + API con Swagger + Postman + PostgreSQL + auth + negocio + harness de tests.
MVP	Construir cliente → crédito → cuota → cobrador → cobro → cartera.
Piloto	Operar con 1–3 negocios y medir errores y tiempos.
V1	Reportes, soporte, notificaciones y experiencia móvil.
Escalamiento	Automatizar despliegue, monitoreo, backups y soporte.

Los tiempos específicos se definirán en la planeación técnica.

19. Criterios de aceptación y pendientes

Criterios de aceptación del MVP

- Un negocio puede registrarse y acceder de forma segura.
- Puede crear cobradores y clientes.
- Puede crear un crédito y consultar sus cuotas.
- Puede asignar cartera.
- El cobrador ve sus cobros del día.
- Puede registrar un pago.
- Saldo y estado se actualizan correctamente (sin estados parciales).
- Administrador consulta cartera y recaudo.
- Operaciones críticas quedan auditadas.
- No existe acceso entre negocios (`negocio_id`).
- El flujo principal funciona correctamente desde teléfono (Angular responsive).
- `docker compose up` levanta web, api, workers y base de datos.
- Swagger UI documenta los endpoints del núcleo.
- La colección Postman cubre autenticación y el flujo cliente → crédito → cobro.
- Los tests automatizados del núcleo (auth, clientes, cobro) pasan en verde.

Pendientes antes del desarrollo detallado

- Campos obligatorios de cliente.
- Identificación y validaciones.
- Tipos de crédito.
- Cálculo exacto de tasa/interés/porcentaje.
- Periodicidades definitivas.
- Redondeos.
- Mora y recargos.
- Pagos parciales, anticipados o superiores.
- Refinanciación/reestructuración.
- Anulación/corrección de pagos (política comercial; el modelo técnico ya contempla anulación con motivo).
- Métodos de pago.
- Comprobantes.
- Modo offline.

- WhatsApp/SMS.
- Política de suspensión/cancelación.
- SLA y soporte.
- Requisitos legales y tributarios.